



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 4 de enero
de 2026

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	1
Apéndice B Especificación de Requisitos	3
B.1. Introducción	3
B.2. Objetivos generales	3
B.3. Catálogo de requisitos	3
B.4. Especificación de requisitos	3
Apéndice C Especificación de diseño	5
C.1. Introducción	5
C.2. Diseño de datos	5
C.3. Diseño arquitectónico	11
C.4. Diseño procedimental	12
Apéndice D Documentación técnica de programación	13
D.1. Introducción	13
D.2. Estructura de directorios	13
D.3. Manual del programador	13

D.4. Compilación, instalación y ejecución del proyecto	13
D.5. Pruebas del sistema	13
Apéndice E Documentación de usuario	15
E.1. Introducción	15
E.2. Requisitos de usuarios	15
E.3. Instalación	15
E.4. Manual del usuario	15
Apéndice F Anexo de sostenibilización curricular	17
F.1. Introducción	17
Bibliografía	19

Índice de figuras

Índice de tablas

C.1. Comparativa de ventajas e inconvenientes entre Bootstrap y Tailwind CSS	7
C.2. Comparativa de motores y fuentes de datos geoespaciales	9

Apéndice A

Plan de Proyecto Software

A.1. Introducción

A.2. Planificación temporal

A.3. Estudio de viabilidad

Viabilidad económica

Viabilidad legal

Apéndice B

Especificación de Requisitos

B.1. Introducción

Una muestra de cómo podría ser una tabla de casos de uso:

B.2. Objetivos generales

B.3. Catálogo de requisitos

B.4. Especificación de requisitos

Apéndice C

Especificación de diseño

C.1. Introducción

C.2. Diseño de datos

Comparativa entre Bootstrap y Tailwind CSS

En el diseño e implementación de la interfaz web del proyecto resulta necesario seleccionar un *framework* de estilos que aporte productividad sin comprometer la mantenibilidad ni la calidad del código. Entre las alternativas más consolidadas se encuentran Bootstrap, orientado a componentes predefinidos, y Tailwind CSS, basado en un paradigma de utilidades. A continuación se presenta una comparativa crítica entre ambas opciones, justificando la elección final de Tailwind CSS para el desarrollo de esta página.

Bootstrap propone un conjunto amplio de componentes listos para usar, tales como barras de navegación, tarjetas o sistemas de rejilla, acompañados de un diseño visual relativamente homogéneo. Esta aproximación permite, en un primer momento, acelerar el desarrollo, ya que se pueden componer interfaces funcionales mediante la combinación de bloques estándar. Además, Bootstrap ofrece una documentación madura y una comunidad extensa, lo que facilita la resolución de dudas y la búsqueda de ejemplos; sin embargo, esta misma estandarización introduce ciertas limitaciones. Por un lado, la personalización profunda del diseño suele requerir sobrescribir reglas de estilo, lo que incrementa la complejidad de las hojas CSS y genera deuda técnica con facilidad. Por otro lado, las interfaces tienden a parecerse entre

sí, de forma que la identidad visual del proyecto puede quedar diluida si no se realiza un esfuerzo considerable de adaptación.

En contraste, Tailwind CSS se basa en clases utilitarias de bajo nivel que describen propiedades concretas, tales como márgenes, tipografía o colores, aplicadas directamente sobre el marcado HTML. Esta filosofía permite construir componentes a medida a partir de combinaciones de utilidades, sin depender de un conjunto rígido de componentes predefinidos. Como consecuencia, el diseño resultante se ajusta mejor a los requisitos específicos del proyecto y favorece la creación de un sistema de diseño propio, más alineado con la identidad visual deseada. Además, Tailwind CSS integra mecanismos de purga de clases no utilizadas, lo que reduce de manera significativa el tamaño del fichero CSS final y mejora el rendimiento en producción.

No obstante, Tailwind CSS presenta también ciertos inconvenientes. Al inicio, la curva de aprendizaje puede parecer más pronunciada, ya que obliga a pensar en términos de propiedades de bajo nivel y no tanto en componentes cerrados. Asimismo, la presencia de múltiples clases en cada elemento puede dar lugar a un marcado aparentemente más denso, pero, cuando se interioriza el conjunto de utilidades y se adoptan buenas prácticas, como la extracción de componentes reutilizables y el uso de configuración centralizada, el resultado es un código más predecible y coherente, con menos necesidad de sobrescrituras complejas.

Desde una perspectiva de la ingeniería del software, la elección entre ambos enfoques afecta de forma directa a la mantenibilidad y escalabilidad del proyecto. Bootstrap ofrece rapidez inicial, pero puede derivar en hojas de estilo difíciles de gestionar cuando se requiere un alto grado de personalización. En cambio, Tailwind CSS promueve una relación más estrecha entre el diseño y el código, ya que el sistema de diseño se expresa mediante utilidades consistentes que se reutilizan en toda la aplicación. Esto reduce la duplicidad de estilos, minimiza el riesgo de efectos colaterales al modificar clases existentes y facilita la evolución del producto a largo plazo.

Finalmente, he optado por Tailwind CSS debido a su capacidad para generar interfaces más ligeras, a su enfoque modular y a su mejor integración con patrones modernos de desarrollo basado en componentes. Gracias a su sistema de utilidades, ha sido posible definir un diseño adaptado a las necesidades concretas del proyecto, mantener un control fino sobre el *bundle* de estilos y reducir la deuda técnica asociada a la personalización del *frontend*. En definitiva, Tailwind CSS proporciona un equilibrio más adecuado entre

flexibilidad, rendimiento y mantenibilidad, lo que lo convierte en la opción más idónea para este desarrollo.

Aspecto	Bootstrap
Enfoque	Conjunto de componentes predefinidos y est
Curva de aprendizaje	Accesible al inicio gracias a los componentes estándar y
Productividad inicial	Elevada en prototipos y desarrollos con requisitos
Personalización del diseño	Personalización profunda costosa, necesidad frecuente de sobresc
Mantenibilidad	Riesgo de hojas CSS voluminosas y difíciles de refactorizar
Rendimiento y tamaño del CSS	Tamaño mayor en el fichero final cuando se utilizan muchos com
Identidad visual	Interfaces con aspecto reconocible y similar a otros proyectos
Comunidad y ecosistema	Comunidad muy amplia, gran cantidad de plantillas, eje

Tabla C.1: Comparativa de ventajas e inconvenientes entre Bootstrap y Tailwind CSS

Comparativa de motores y fuentes de datos geoespaciales

La elección del motor geoespacial y de la fuente de datos constituye una decisión crítica en el diseño de aplicaciones web orientadas al análisis territorial y a la detección de elementos lineales, ya que condiciona de forma directa tanto el nivel de detalle alcanzable como la escalabilidad, la reproducibilidad y la viabilidad legal del sistema. En este contexto, se han analizado cinco alternativas representativas del estado del arte actual, valorando sus capacidades técnicas, sus limitaciones y su adecuación a los objetivos del proyecto.

En primer lugar, **:contentReference[oaicite:1]index=1** se presenta como una plataforma de computación geoespacial en la nube que destaca por su amplio catálogo de datos científicos y por su capacidad de procesado a gran escala. Permite trabajar de forma eficiente con series temporales extensas y con colecciones como Sentinel-2 o Landsat, facilitando la generación de mosaicos, máscaras de nubes o índices espectrales. No obstante, su principal limitación en este proyecto radica en la resolución espacial de dichas fuentes, que suele resultar insuficiente para la detección de trazas finas, como senderos o caminos estrechos. Además, conviene subrayar que la imagerie de alta resolución visible en Google Earth no forma parte de los datasets públicos

accesibles para análisis y exportación, lo que restringe su uso como fuente principal de detalle.

Como alternativa orientada a la integración en aplicaciones web, **:contentReference[oaicite:2]index=2** y el Copernicus Data Space ofrecen interfaces de programación que permiten solicitar imágenes procesadas bajo demanda. Esta aproximación resulta especialmente cómoda en arquitecturas cliente-servidor, ya que simplifica el flujo de petición y respuesta mediante recortes espaciales y temporales. Sin embargo, al igual que en el caso anterior, la resolución de las imágenes Sentinel constituye un factor limitante cuando se requiere un nivel de detalle submétrico, lo que reduce su idoneidad para el objetivo concreto del proyecto.

Una tercera opción, de especial relevancia en el ámbito nacional, es el uso de ortofotos aéreas de alta resolución, destacando en España el Plan Nacional de Ortofotografía Aérea, **:contentReference[oaicite:3]index=3**. Este conjunto de datos proporciona imágenes con resoluciones del orden de decímetros, muy superiores a las de los sensores satelitales de libre acceso. Además, su disponibilidad a través de servicios WMS y WMTS, junto con opciones de descarga directa, lo convierte en una solución sólida tanto para la visualización como para el análisis reproducible. Desde una perspectiva académica, el uso de fuentes oficiales y abiertas refuerza la validez metodológica del trabajo y evita problemas derivados de licencias restrictivas.

En contraposición, el uso directo de imagerie procedente de **:contentReference[oaicite:4]index=4** o Google Earth como fuente de datos para análisis automático presenta importantes inconvenientes. A pesar de su elevada calidad visual, estas plataformas imponen restricciones estrictas en cuanto a la extracción, almacenamiento y procesamiento de imágenes, lo que dificulta su empleo como motor principal en un proyecto de carácter académico y analítico. Por este motivo, su utilización queda prácticamente relegada a contextos de visualización, no siendo recomendable como base para flujos de detección y análisis a escala.

Finalmente, en lo relativo a la capa de visualización, se ha considerado el uso de **:contentReference[oaicite:5]index=5** frente a la integración directa de bibliotecas JavaScript como **:contentReference[oaicite:6]index=6**. Aunque Folium resulta adecuado para prototipos rápidos generados desde Python, su naturaleza estática y su dependencia del backend lo hacen menos apropiado para aplicaciones web interactivas complejas. En consecuencia, una solución basada en Leaflet u OpenLayers en el frontend, combinada

con un backend que sirva datos vectoriales en formato JSON, ofrece mayor flexibilidad y control.

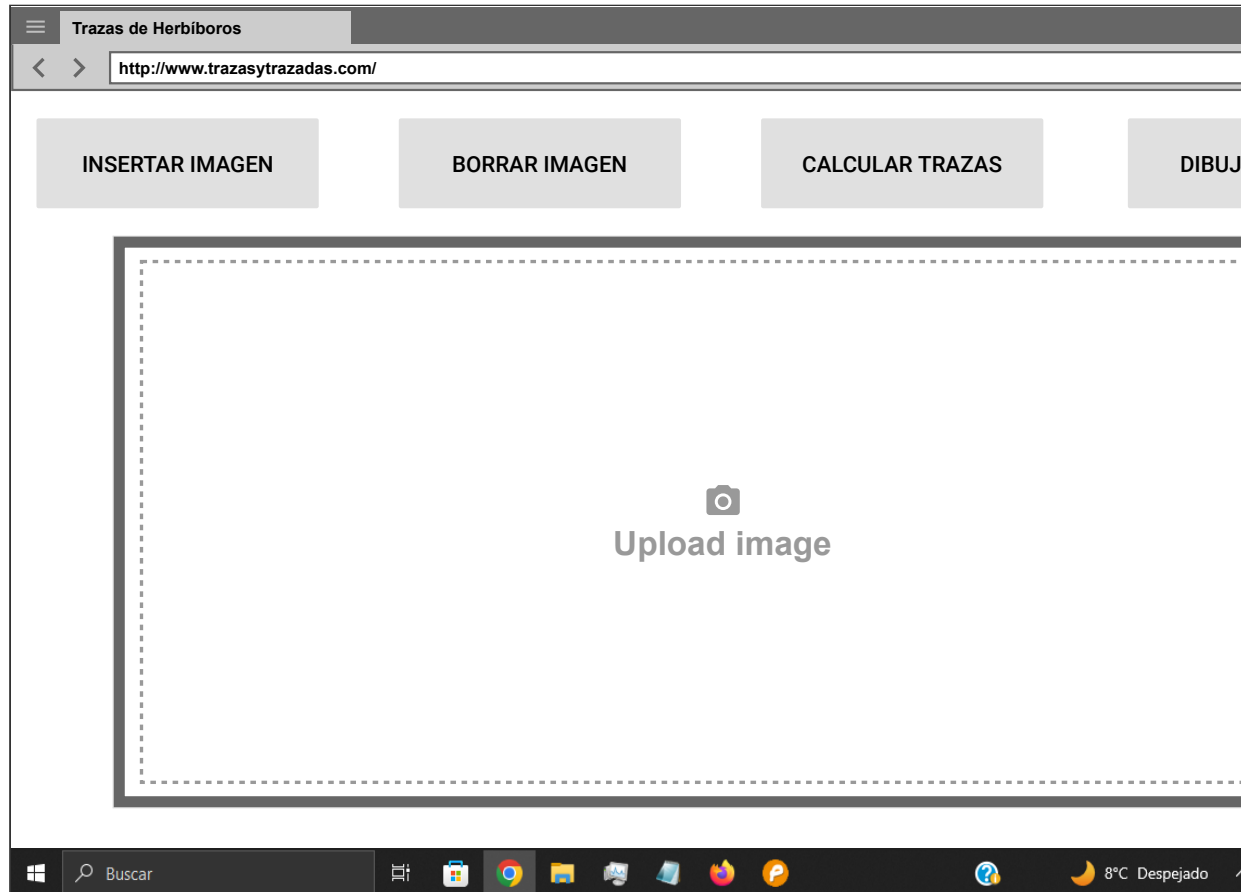
A la luz de este análisis comparativo, la solución adoptada en el proyecto consiste en utilizar ortofotografía aérea de alta resolución, concretamente PNOA, como motor principal de datos geoespaciales. Esta elección garantiza el nivel de detalle necesario para la detección de trazas finas, asegura la reproducibilidad del trabajo y se ajusta a criterios de legalidad y sostenibilidad académica. De forma complementaria, se contempla el uso de Google Earth Engine como herramienta de preprocesado o filtrado a gran escala, aprovechando su potencia para análisis masivos sin depender de él como fuente final de detalle. Este enfoque híbrido proporciona un equilibrio óptimo entre precisión espacial, escalabilidad y rigor metodológico.

Alternativa	
Google Earth Engine	Gran catálogo de datos c
Sentinel Hub / Copernicus	Integración sen
Ortofoto aérea (PNOA)	Alta resolución submét
Google Maps / Earth imagery	
Folium frente a Leaflet	

Tabla C.2: Comparativa de motores y fuentes de datos geoespaciales

C.3. Diseño arquitectónico

Página Inicial



C.4. Diseño procedimental

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía
